Department of CSE, PES University

UE24CS341A: SOFTWARE ENGINEERING

5TH SEMESTER, ACADEMIC YEAR 2026-27

LAB 6: CODE AND BRANCH COVERAGE ANALYSIS DATE: 28-10-2026

NAME: PRATHAM KUMAR

SRN NO.: PES2UG24CS368

PRN NO.: PES2202400086

ASSIGNMENT WORK

Objective

- Understand the difference between code coverage and branch coverage.
- Use coverage to measure, analyse, and improve test coverage.
- Interpret coverage reports and identify untested paths in code.
- Practice writing test cases that cover edge cases and decision branches.

Duration: 60 minutes

Software Requirements

- Python 3.x
- coverage (install via pip)
- Any code editor (VS Code / PyCharm / Sublime / etc.)
- pytest (install via pip)

Coverage can be used standalone or with test frameworks.

Learning Outcomes

- Understand Code Coverage Concepts: You will differentiate between line coverage, branch coverage, path coverage, and function coverage to measure test completeness.
- Apply Coverage Analysis Tools: You will use Python's coverage library to generate reports, interpret coverage metrics, and identify untested code paths.
- Design Comprehensive Test Cases: You will write targeted test cases to achieve minimum 90% to 100% coverage by addressing edge cases and decision branches.

Deliverables:

- Coverage (Code and Branch) analysis (Terminal Output Screenshots)
- CodeCoverageTest_xxxx_application.py (test cases for code coverage)
- BranchCoverageTest_xxxx_application.py (test cases for branch coverage)
- Minimum 2 commits. One for updated code coverage test case, and other for branch coverage test case.

Department of CSE, PES University

UE24CS341A: SOFTWARE ENGINEERING
5TH SEMESTER, ACADEMIC YEAR 2026-27
LAB 6: CODE AND BRANCH COVERAGE ANALYSIS DATE: 28-10-2026

1. Software and Environment Setup

The following software and tools were used to perform the experiment:

Component	Version / Details
Operating System	Microsoft Windows
Programming Language	Python 3.14.0
Package Manager	pip 26.2.1
Testing Framework	pytest 9.2.0
Coverage Analysis Tool	coverage.py 7.17.0
Version Control System	Git
Repository Hosting	GitHub
Code Execution Environment	Windows PowerShell

1.1. Dependency Verification

The required testing dependencies were verified using the following commands:

- python -m pip install pytest coverage
- python -m pytest --version
- python -m coverage --version

The installation command reported that both packages were already installed in the Python environment. Their versions were subsequently verified successfully.

- pytest 9.2.0 was available for executing test cases.
- coverage.py 7.17.0 was available for measuring code coverage and branch coverage.

1.2. Repository Setup

The faculty-provided starter repository was forked into the student's GitHub account and cloned to the local system.

GitHub repository: https://github.com/PKumar041104/L6_L7_60_marathon_registration
The repository was cloned using: git clone
https://github.com/PKumar041104/L6_L7_60_marathon_registration.git
cd L6_L7_60_marathon_registration

The Lab 6 implementation and testing files were located in the LAB6 directory.

File	Purpose
marathon_registration.py	Application module containing the functions under test
test_marathon_registration.py	Starter test file supplied with the repository
README.md	Project description and instructions

The Git repository was verified to be connected to the student's fork, and the initial working tree was clean.

2. Initial Test Execution and Baseline Coverage

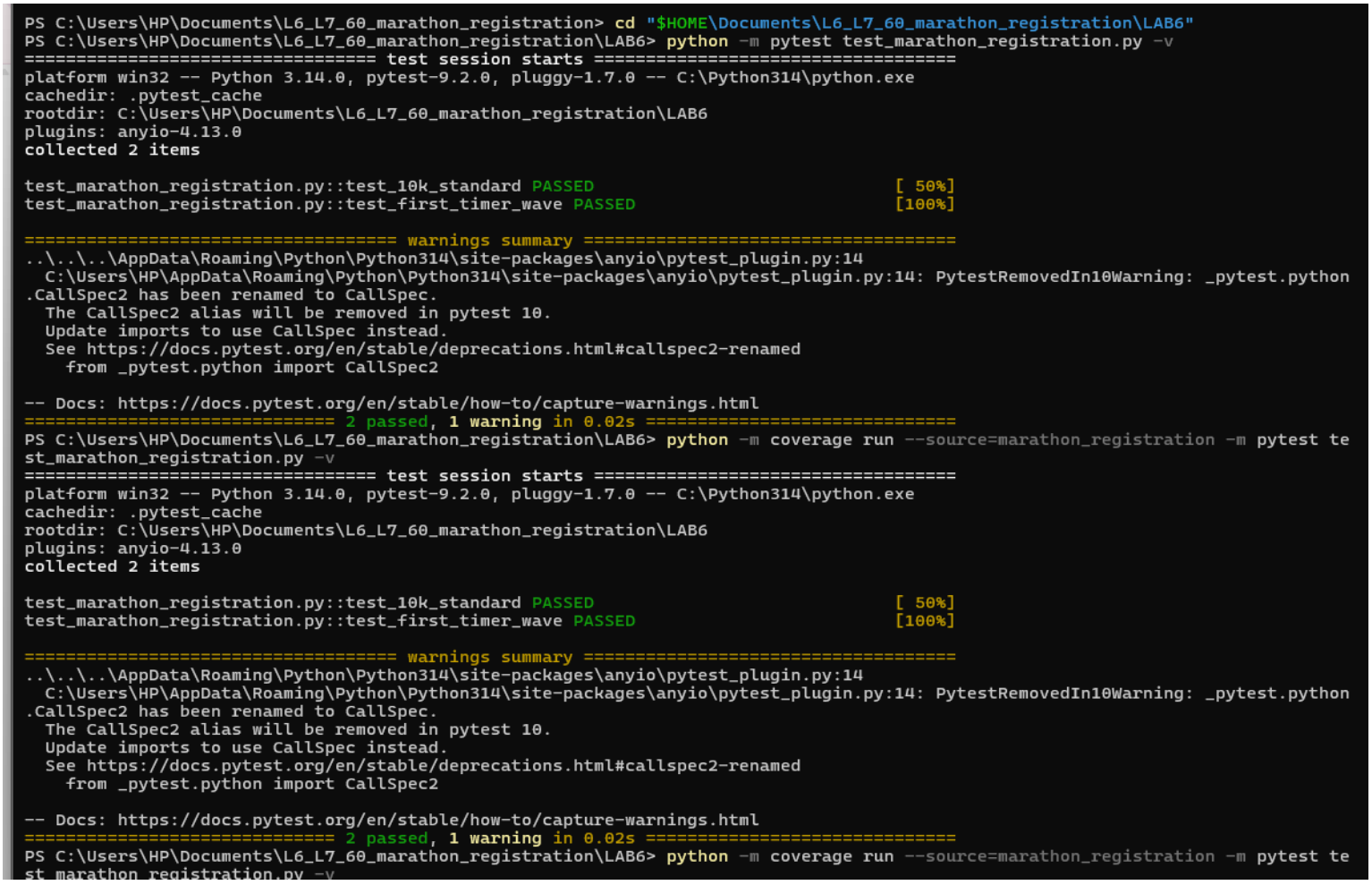
2.1. Execution of Starter Test Cases

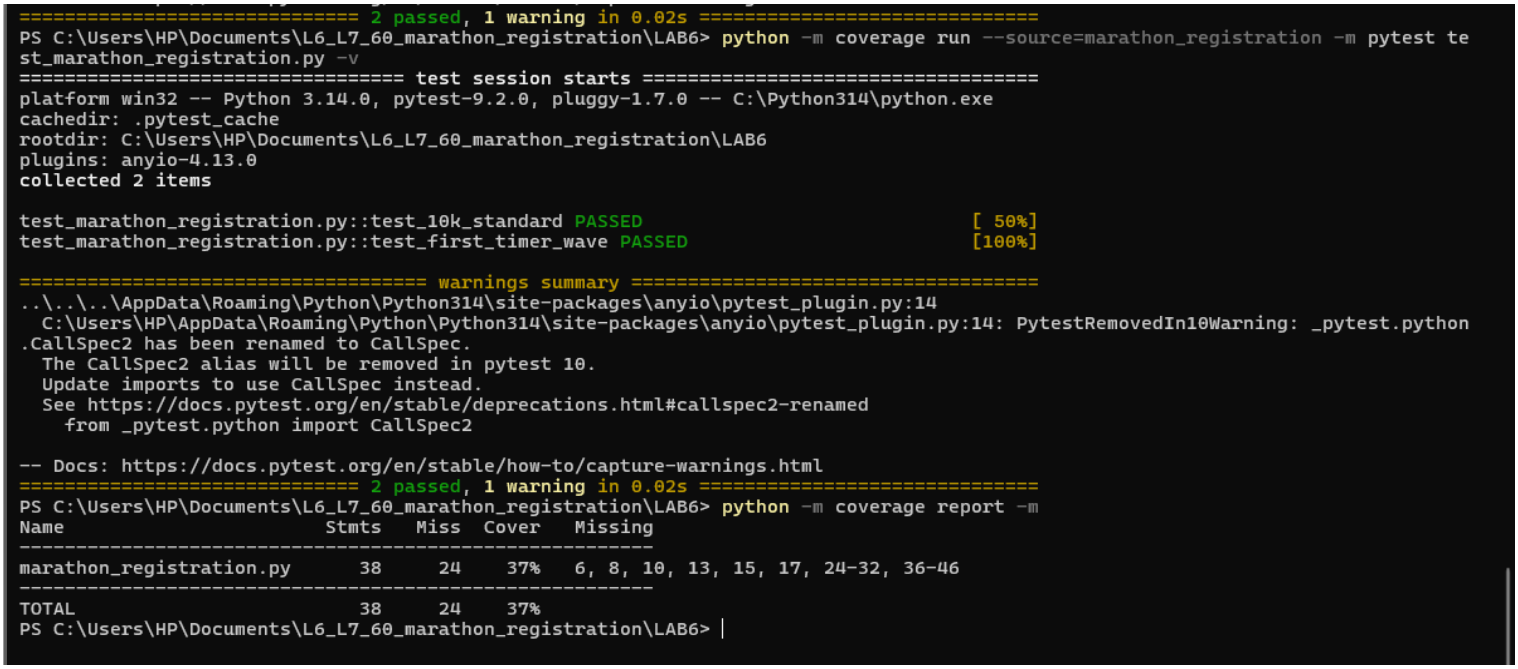
The original test file was executed before adding new test cases.
Command: python -m pytest test_marathon_registration.py -v
The test execution completed successfully. Both starter tests passed:

1. test_10k_standard — verified the standard entry fee for a 10K race.
2. test_first_timer_wave — verified that a runner without a recorded best time was assigned to start wave D.

The output also displayed a deprecation warning originating from a pytest dependency. This warning did not affect the test results.

Figure 1: Initial Execution of Starter Test Cases





Explanation: Figure 1 demonstrates that the supplied test cases execute successfully. However, passing the existing tests does not establish that all application logic has been tested. Additional tests are therefore required to exercise untested statements and decision outcomes.

2.2. Baseline Code Coverage Analysis

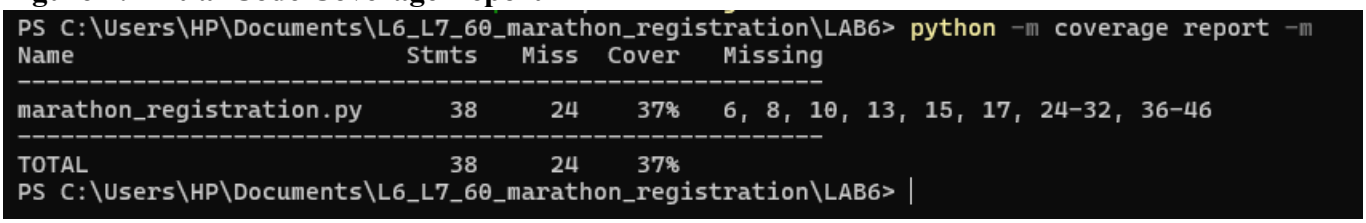
The baseline coverage measurement was performed using the following commands:

python -m coverage run --source=marathon_registration -m pytest test_marathon_registration.py -v
python -m coverage report -m

The coverage report produced the following results:

Metric	Initial Result
Statements	38
Missing statements	24
Line coverage	37%

Figure 2: Initial Code Coverage Report



Explanation: Figure 2 shows that the two starter tests executed only a limited portion of the application. Of the 38 measured statements, 24 were not executed, resulting in 37% line coverage. The missing statements indicate that important validation rules, pricing conditions, start-wave decisions, and finishing-status outcomes require additional testing.

2.3. Baseline Findings

The initial analysis established the need to expand the test suite. The next stage will introduce a separate test file for code coverage, targeting the application's entry-fee calculations, start-wave assignments, validation errors, and finisher-status rules. After achieving and verifying at least 90% line coverage, a separate branch-coverage stage will test the alternative outcomes of decision statements.

3. Code Coverage Analysis

3.1. Test Case Development

A separate test file, CodeCoverageTest_PES2UG24CS368_marathon_registration.py, was created to expand the test suite without modifying the original application code.
The test cases cover standard entry fees for different race categories, invalid race types, age restrictions, early-bird discounts, senior discounts, T-shirt charges, start-wave assignments, missed checkpoints, and finishing-time limits. Parameterized tests were used to test multiple input combinations efficiently.

3.2. Execution and Results

The expanded test suite was executed using the pytest framework, and code coverage was measured using coverage.py.
Commands used:
python -m coverage erase
python -m coverage run --source=marathon_registration -m pytest CodeCoverageTest_PES2UG24CS368_marathon_registration.py -v
python -m coverage report -m

The execution completed with the following results:

Metric	Result
Total test cases	35
Passed test cases	35
Failed test cases	0
Total measured statements	38
Missing statements	0
Code coverage	100%

Figure 3: Terminal Output Showing 100% Code Coverage

```
PS C:\Users\HP\Documents\L6_L7_60_marathon_registration\LAB6> notepad CodeCoverageTest_PES2UG24CS368_marathon_registration.py
PS C:\Users\HP\Documents\L6_L7_60_marathon_registration\LAB6> notepad BranchCoverageTest_PES2UG24CS368_marathon_registration.py
PS C:\Users\HP\Documents\L6_L7_60_marathon_registration\LAB6> python -m coverage erase
PS C:\Users\HP\Documents\L6_L7_60_marathon_registration\LAB6> python -m coverage run --source=marathon_registration -m pytest CodeCover
ageTest_PES2UG24CS368_marathon_registration.py -v
========================================= test session starts =========================================
platform win32 -- Python 3.14.0, pytest-9.2.0, pluggy-1.7.0 -- C:\Python314\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\HP\Documents\L6_L7_60_marathon_registration\LAB6
plugins: anyio-4.13.0
collected 35 items

CodeCoverageTest_PES2UG24CS368_marathon_registration.py::test_standard_entry_fees[5k-500] PASSED [  2%]
CodeCoverageTest_PES2UG24CS368_marathon_registration.py::test_standard_entry_fees[10k-800] PASSED [  5%]
CodeCoverageTest_PES2UG24CS368_marathon_registration.py::test_standard_entry_fees[half-1200] PASSED [  8%]
CodeCoverageTest_PES2UG24CS368_marathon_registration.py::test_standard_entry_fees[full-1800] PASSED [ 11%]
CodeCoverageTest_PES2UG24CS368_marathon_registration.py::test_unknown_race PASSED            [ 14%]
CodeCoverageTest_PES2UG24CS368_marathon_registration.py::test_age_below_minimum PASSED       [ 17%]
CodeCoverageTest_PES2UG24CS368_marathon_registration.py::test_underage_long_distance[half] PASSED [ 20%]
CodeCoverageTest_PES2UG24CS368_marathon_registration.py::test_underage_long_distance[full] PASSED [ 22%]
CodeCoverageTest_PES2UG24CS368_marathon_registration.py::test_age_12_is_accepted_for_5k PASSED [ 25%]
CodeCoverageTest_PES2UG24CS368_marathon_registration.py::test_age_18_is_accepted_for_half_marathon PASSED [ 28%]
CodeCoverageTest_PES2UG24CS368_marathon_registration.py::test_early_bird_discount PASSED     [ 31%]
CodeCoverageTest_PES2UG24CS368_marathon_registration.py::test_senior_discount_at_65 PASSED   [ 34%]
CodeCoverageTest_PES2UG24CS368_marathon_registration.py::test_tshirt_surcharge PASSED        [ 37%]
CodeCoverageTest_PES2UG24CS368_marathon_registration.py::test_combined_discounts_and_tshirt PASSED [ 40%]
CodeCoverageTest_PES2UG24CS368_marathon_registration.py::test_early_bird_discount_rounding PASSED [ 42%]
CodeCoverageTest_PES2UG24CS368_marathon_registration.py::test_no_discount_for_age_64 PASSED  [ 45%]
CodeCoverageTest_PES2UG24CS368_marathon_registration.py::test_start_wave_without_time PASSED [ 48%]
CodeCoverageTest_PES2UG24CS368_marathon_registration.py::test_start_wave_decisions[180-full-A] PASSED [ 51%]
CodeCoverageTest_PES2UG24CS368_marathon_registration.py::test_start_wave_decisions[181-full-B] PASSED [ 54%]
CodeCoverageTest_PES2UG24CS368_marathon_registration.py::test_start_wave_decisions[240-full-B] PASSED [ 57%]
CodeCoverageTest_PES2UG24CS368_marathon_registration.py::test_start_wave_decisions[241-full-C] PASSED [ 60%]
CodeCoverageTest_PES2UG24CS368_marathon_registration.py::test_start_wave_decisions[90-10k-A] PASSED [ 62%]
CodeCoverageTest_PES2UG24CS368_marathon_registration.py::test_start_wave_decisions[91-10k-B] PASSED [ 65%]
CodeCoverageTest_PES2UG24CS368_marathon_registration.py::test_start_wave_decisions[120-10k-B] PASSED [ 68%]
CodeCoverageTest_PES2UG24CS368_marathon_registration.py::test_start_wave_decisions[121-10k-C] PASSED [ 71%]
CodeCoverageTest_PES2UG24CS368_marathon_registration.py::test_missed_checkpoint_disqualifies_runner PASSED [ 74%]
CodeCoverageTest_PES2UG24CS368_marathon_registration.py::test_multiple_missed_checkpoints_disqualify_runner PASSED [ 77%]
CodeCoverageTest_PES2UG24CS368_marathon_registration.py::test_finisher_status_time_limits[full-360-finisher] PASSED [ 80%]
CodeCoverageTest_PES2UG24CS368_marathon_registration.py::test_finisher_status_time_limits[full-361-DNF] PASSED [ 82%]
CodeCoverageTest_PES2UG24CS368_marathon_registration.py::test_finisher_status_time_limits[half-180-finisher] PASSED [ 85%]
CodeCoverageTest_PES2UG24CS368_marathon_registration.py::test_finisher_status_time_limits[half-181-DNF] PASSED [ 88%]
CodeCoverageTest_PES2UG24CS368_marathon_registration.py::test_finisher_status_time_limits[5k-90-finisher] PASSED [ 91%]
CodeCoverageTest_PES2UG24CS368_marathon_registration.py::test_finisher_status_time_limits[5k-91-DNF] PASSED [ 94%]
CodeCoverageTest_PES2UG24CS368_marathon_registration.py::test_finisher_status_time_limits[10k-90-finisher] PASSED [ 97%]
CodeCoverageTest_PES2UG24CS368_marathon_registration.py::test_finisher_status_time_limits[10k-91-DNF] PASSED [100%]

========================================= warnings summary =========================================
..\..\..\AppData\Roaming\Python\Python314\site-packages\anyio\pytest_plugin.py:14
  C:\Users\HP\AppData\Roaming\Python\Python314\site-packages\anyio\pytest_plugin.py:14: PytestRemovedIn10Warning: _pytest.python.CallSp
ec2 has been renamed to CallSpec.
  The CallSpec2 alias will be removed in pytest 10.
  Update imports to use CallSpec instead.
  See https://docs.pytest.org/en/stable/deprecations.html#callspec2-renamed
    from _pytest.python import CallSpec2

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
================================ 35 passed, 1 warning in 0.07s ================================
PS C:\Users\HP\Documents\L6_L7_60_marathon_registration\LAB6> python -m coverage report -m
Name                     Stmts   Miss  Cover   Missing
```

```
PS C:\Users\HP\Documents\L6_L7_60_marathon_registration\LAB6> python -m coverage report -m
Name                       Stmts   Miss  Cover   Missing
--------------------------------------------------------
marathon_registration.py      38      0   100%
--------------------------------------------------------
TOTAL                         38      0   100%
PS C:\Users\HP\Documents\L6_L7_60_marathon_registration\LAB6>
```

Explanation: Figure 3 demonstrates that all 35 test cases passed successfully. The coverage report identifies 38 measured statements and zero missing statements, resulting in 100% line coverage. This exceeds the minimum coverage requirement of 90%. The result indicates that every measured executable statement in the application was executed by the code-coverage test suite.

3.3. Observation

The additional test cases exercised the application's major functional operations, validation rules, and boundary conditions. The achieved 100% line coverage provides evidence that every measured statement was executed during testing. However, line coverage alone does not establish that every decision outcome has been tested; branch coverage must therefore be measured separately.

4. Branch Coverage Analysis

4.1. Test Case Development

A separate test file, BranchCoverageTest_PES2UG24CS368_marathon_registration.py, was created to exercise the alternative outcomes of the application's decision statements.
The tests cover valid and invalid race entries, age restrictions, discount conditions, start-wave assignments, checkpoint disqualification, and finishing-time decisions. Boundary values were included to verify outcomes at and around the defined thresholds.

4.2. Execution and Results

Branch coverage was measured by running the branch-coverage test suite with the --branch option.
Commands used:
python -m coverage erase
python -m coverage run --branch --source=marathon_registration -m pytest BranchCoverageTest_PES2UG24CS368_marathon_registration.py -v
python -m coverage report -m
python -m coverage json -o branch_coverage.json
The generated coverage data was then inspected to calculate the branch-specific percentage.

Metric	Result
Total test cases	28
Passed test cases	28
Failed test cases	0
Total measured statements	38
Missing statements	0
Total branches	28
Missing branches	0
Partially covered branches	0
Line coverage	100%
Branch coverage	100%

Figure 4: Terminal Output Showing 100% Branch Coverage

```
PS C:\Users\HP\Documents\L6_L7_60_marathon_registration\LAB6> python -m coverage erase
PS C:\Users\HP\Documents\L6_L7_60_marathon_registration\LAB6> python -m coverage run --branch --source=marathon_registration -m pytest
BranchCoverageTest_PES2UG24CS368_marathon_registration.py -v
============================================================ test session starts ============================================================
platform win32 -- Python 3.14.0, pytest-9.2.0, pluggy-1.7.0 -- C:\Python314\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\HP\Documents\L6_L7_60_marathon_registration\LAB6
plugins: anyio-4.13.0
collected 28 items

BranchCoverageTest_PES2UG24CS368_marathon_registration.py::test_entry_fee_valid_paths[5k-30-False-False-500] PASSED              [  3%]
BranchCoverageTest_PES2UG24CS368_marathon_registration.py::test_entry_fee_valid_paths[10k-30-True-False-600] PASSED              [  7%]
BranchCoverageTest_PES2UG24CS368_marathon_registration.py::test_entry_fee_valid_paths[half-30-False-True-1500] PASSED            [ 10%]
BranchCoverageTest_PES2UG24CS368_marathon_registration.py::test_entry_fee_valid_paths[full-70-True-True-975] PASSED              [ 14%]
BranchCoverageTest_PES2UG24CS368_marathon_registration.py::test_entry_fee_valid_paths[5k-65-False-False-250] PASSED              [ 17%]
BranchCoverageTest_PES2UG24CS368_marathon_registration.py::test_entry_fee_valid_paths[10k-65-True-False-300] PASSED              [ 21%]
BranchCoverageTest_PES2UG24CS368_marathon_registration.py::test_entry_fee_unknown_race_branch PASSED                             [ 25%]
BranchCoverageTest_PES2UG24CS368_marathon_registration.py::test_entry_fee_under_12_branch PASSED                                 [ 28%]
BranchCoverageTest_PES2UG24CS368_marathon_registration.py::test_entry_fee_short_race_skips_long_distance_age_check PASSED       [ 32%]
BranchCoverageTest_PES2UG24CS368_marathon_registration.py::test_entry_fee_long_race_under_18_branch[half] PASSED                 [ 35%]
BranchCoverageTest_PES2UG24CS368_marathon_registration.py::test_entry_fee_long_race_under_18_branch[full] PASSED                 [ 39%]
BranchCoverageTest_PES2UG24CS368_marathon_registration.py::test_entry_fee_long_race_age_18_branch PASSED                         [ 42%]
BranchCoverageTest_PES2UG24CS368_marathon_registration.py::test_start_wave_branches[None-full-D] PASSED                          [ 46%]
BranchCoverageTest_PES2UG24CS368_marathon_registration.py::test_start_wave_branches[180-full-A] PASSED                           [ 50%]
BranchCoverageTest_PES2UG24CS368_marathon_registration.py::test_start_wave_branches[200-full-B] PASSED                           [ 53%]
BranchCoverageTest_PES2UG24CS368_marathon_registration.py::test_start_wave_branches[241-full-C] PASSED                           [ 57%]
BranchCoverageTest_PES2UG24CS368_marathon_registration.py::test_start_wave_branches[90-10k-A] PASSED                             [ 60%]
BranchCoverageTest_PES2UG24CS368_marathon_registration.py::test_start_wave_branches[100-10k-B] PASSED                            [ 64%]
BranchCoverageTest_PES2UG24CS368_marathon_registration.py::test_start_wave_branches[121-10k-C] PASSED                            [ 67%]
BranchCoverageTest_PES2UG24CS368_marathon_registration.py::test_finisher_disqualification_branch PASSED                          [ 71%]
BranchCoverageTest_PES2UG24CS368_marathon_registration.py::test_finisher_time_branches[full-360-finisher] PASSED                 [ 75%]
BranchCoverageTest_PES2UG24CS368_marathon_registration.py::test_finisher_time_branches[full-361-DNF] PASSED                      [ 78%]
BranchCoverageTest_PES2UG24CS368_marathon_registration.py::test_finisher_time_branches[half-180-finisher] PASSED                 [ 82%]
BranchCoverageTest_PES2UG24CS368_marathon_registration.py::test_finisher_time_branches[half-181-DNF] PASSED                      [ 85%]
BranchCoverageTest_PES2UG24CS368_marathon_registration.py::test_finisher_time_branches[5k-90-finisher] PASSED                    [ 89%]
BranchCoverageTest_PES2UG24CS368_marathon_registration.py::test_finisher_time_branches[5k-91-DNF] PASSED                         [ 92%]
BranchCoverageTest_PES2UG24CS368_marathon_registration.py::test_finisher_time_branches[10k-90-finisher] PASSED                   [ 96%]
BranchCoverageTest_PES2UG24CS368_marathon_registration.py::test_finisher_time_branches[10k-91-DNF] PASSED                        [100%]

============================================================ warnings summary ============================================================
..\..\..\AppData\Roaming\Python\Python314\site-packages\anyio\pytest_plugin.py:14
  C:\Users\HP\AppData\Roaming\Python\Python314\site-packages\anyio\pytest_plugin.py:14: PytestRemovedIn10Warning: _pytest.python.CallSp
ec2 has been renamed to CallSpec.
  The CallSpec2 alias will be removed in pytest 10.
  Update imports to use CallSpec instead.
  See https://docs.pytest.org/en/stable/deprecations.html#callspec2-renamed
    from _pytest.python import CallSpec2

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
============================================ 28 passed, 1 warning in 0.06s ============================================
PS C:\Users\HP\Documents\L6_L7_60_marathon_registration\LAB6> python -m coverage report -m
Name                       Stmts   Miss Branch BrPart  Cover   Missing
----------------------------------------------------------------------
marathon_registration.py      38      0     28      0   100%
----------------------------------------------------------------------
TOTAL                         38      0     28      0   100%
PS C:\Users\HP\Documents\L6_L7_60_marathon_registration\LAB6> python -m coverage json -o branch_coverage.json
Wrote JSON report to branch_coverage.json
PS C:\Users\HP\Documents\L6_L7_60_marathon_registration\LAB6> python -c "import json; d=json.load(open('branch_coverage.json')); f=d['f
iles']['marathon_registration.py']; print('Line coverage:', round(100*f['summary']['percent_covered'], 2), '%'); print('Covered branche
s:', f['summary']['covered_branches']); print('Missing branches:', f['summary']['missing_branches']); print('Branch coverage:', round(1
00*f['summary']['covered_branches']/(f['summary']['covered_branches']+f['summary']['missing_branches']), 2), '%')"
Line coverage: 10000.0 %
Covered branches: 28
Missing branches: 0
Branch coverage: 100.0 %
PS C:\Users\HP\Documents\L6_L7_60_marathon_registration\LAB6>
```

Explanation: Figure 4 demonstrates that all 28 branch-coverage test cases passed. The coverage report identified 28 branches, with no missing or partially covered branches. The branch-specific percentage was calculated from the generated JSON coverage data and confirmed to be 100%. The result exceeds the minimum branch-coverage requirement of 90%.

4.3. Observation

The experiment demonstrates that line coverage and branch coverage measure different aspects of testing. Line coverage measures whether executable statements were run, whereas branch coverage checks whether the alternative outcomes of decision points were exercised. The completed test suites achieved 100% for both reported metrics on the supplied application.

5. GitHub Repository and Version Control

The completed Lab 6 work was maintained in the student's fork of the faculty-provided repository.
GitHub Repository: https://github.com/PKumar041104/L6_L7_60_marathon_registration
All Lab 6 implementation and testing files were maintained within the LAB6 directory.

5.1. Files Submitted

File	Purpose
marathon_registration.py	Original application under test
test_marathon_registration.py	Faculty-provided starter tests
CodeCoverageTest_PES2UG24CS368_marathon_registration.py	Expanded test suite for code coverage
BranchCoverageTest_PES2UG24CS368_marathon_registration.py	Test suite for branch coverage
screenshots/Figure_3_Code_Coverage_100_Percent.png	Evidence of the final code coverage report
screenshots/Figure_4_Branch_Coverage_100_Percent.png	Evidence of the final branch coverage report
.gitignore	Excludes temporary testing and coverage artifacts

5.2. Commit History

Two separate commits were created and pushed successfully to the main branch.

Commit ID	Commit Message
1223f8c	Add code coverage tests and evidence
5e92efe	Add branch coverage tests and evidence

The first commit contains the code-coverage test file and its screenshot evidence. The second commit contains the branch-coverage test file and its screenshot evidence.

Both commits were pushed to the student's GitHub repository. The final Git status confirmed that the local branch was synchronized with origin/main and that the working tree was clean.

6. Final Results and Conclusion

6.1. Summary of Results

Evaluation Metric	Achieved Result
Code-coverage test cases passed	35
Code coverage	100%
Branch-coverage test cases passed	28
Branch coverage	100%
Missing statements	0
Missing branches	0
Partially covered branches	0
Separate Git commits	2
GitHub push	Successful

6.2. Conclusion

This lab demonstrated how code coverage and branch coverage can be used to assess the completeness of a software testing process. The original starter tests passed but achieved only 37% line coverage, indicating that many parts of the application had not been exercised.

Additional test cases were developed for the marathon registration application's fee calculations, validation rules, start-wave assignments, checkpoint disqualification, and finishing-time decisions. The expanded code-coverage suite achieved 100% line coverage, while the branch-coverage suite achieved 100% branch coverage according to the generated coverage data.

The results exceeded the minimum 90% coverage requirement specified in the lab handout. The completed test files and corresponding screenshots were committed separately and pushed to the assigned GitHub fork. The experiment reinforced the importance of testing alternative decision outcomes rather than relying on successful test execution or line coverage alone.

NAME: PRATHAM KUMAR
SRN NO.: PES2UG24CS368
PRN NO.: PES2202400086
SECTION: F

THANK YOU